

KHULNA UNIVERSITY OF ENGINEERING & TECHNOLOGY

Khulna - 9203, Bangladesh



Department of Computer Science and Engineering

Computer Architecture Laboratory (CSE 2114)

Project Report on

23-bit CPU Design

Submission Date : 10 May 2026

Submitted by

Name	: A.T.M Shahnewaj
Roll	: 2307064
Year	: 2nd
Semester	: 1st

Submitted to

Md. Badiuzzaman Shuvo
Lecturer
Department of Computer
Science and Engineering,
KUET

Md. Mubtashim Abrar Nihal
Lecturer (Part-time)
Department of Computer
Science and Engineering,
KUET

1. Project Overview

This project implements a functional 23-bit CPU in Logisim, integrating a custom Arithmetic Logic Unit (ALU), a Control Unit, and a Booth's Multiplication module. The design demonstrates a complete instruction-fetch-decode-execute cycle with support for 13 instructions, including arithmetic, logical, shift, and memory operations.

1.1 Key Specifications

Parameter	Value
Data Width	23 bits
Instruction Set Size	13 instructions (opcodes 0–D hex)
ALU Operations	AND, ADD, OR, SUB, XOR, SHR, ROR, 2's Complement
Memory	RAM with 5-bit address (Addr), 23-bit data bus
Multiplication	Booth's Algorithm (23-bit × 23-bit)
Simulation Tool	Logisim
Clock	Single CLK with synchronous Reset

2. Instruction Set Architecture (ISA)

Instruction Word Format (23 bits)

Each instruction is exactly 23 bits wide. The layout is:

Bits	[22:20]	[19:16]	[15:5]	[4:0]
Field	Don't Care (3 bits)	OPCODE (4 bits)	Don't Care (11 bits)	ADDRESS (5 bits)

The 4-bit opcode supports 16 instruction codes. The 5-bit address field provides 32 memory locations (0x00–0x1F). The Control Unit splitter extracts bits [19:16] from the Instruction Register (IR) as the opcode, which is used to index the microprogram ROM.

The full instruction table is shown below:

Opcode (Hex)	Mnemonic	Operation	Description
0	AND	$AC \leftarrow AC \text{ AND } MBR$	Bitwise AND with Accumulator
1	ADD	$AC \leftarrow AC + MBR$	Add memory operand to AC
2	STO	$MEM[addr] \leftarrow AC$	Store AC to memory
3	OR	$AC \leftarrow AC \text{ OR } MBR$	Bitwise OR with Accumulator
4	SUB	$AC \leftarrow AC - MBR$	Subtract memory operand from AC
5	BUN	$PC \leftarrow addr$	Unconditional branch
6	LDA	$AC \leftarrow MEM[addr]$	Load memory operand into AC
7	HLT	-	Halt execution
8	XOR	$AC \leftarrow AC \text{ XOR } MBR$	Bitwise XOR with Accumulator
9	SHR	$AC \leftarrow AC \gg 1$	Logical right shift
A	ROR	$AC \leftarrow \text{rotate_right}(AC)$	Rotate right through carry
B	2's COMP	$AC \leftarrow \sim AC + 1$	Two's complement negation
C	Upper Booths	$AC \leftarrow \text{BOOTH_HIGH}$	Load upper product of Booth's
D	BOOTH_MUL	Trigger Booth's multiplier	Initiate hardware multiplication

3. System Architecture

The system is organized into three top-level modules: the CPU, the Control Unit, and the Booth's Multiplier. These interact with shared memory and the ALU. The block diagram below summarizes the high-level architecture.

3.1 CPU Internal Datapath

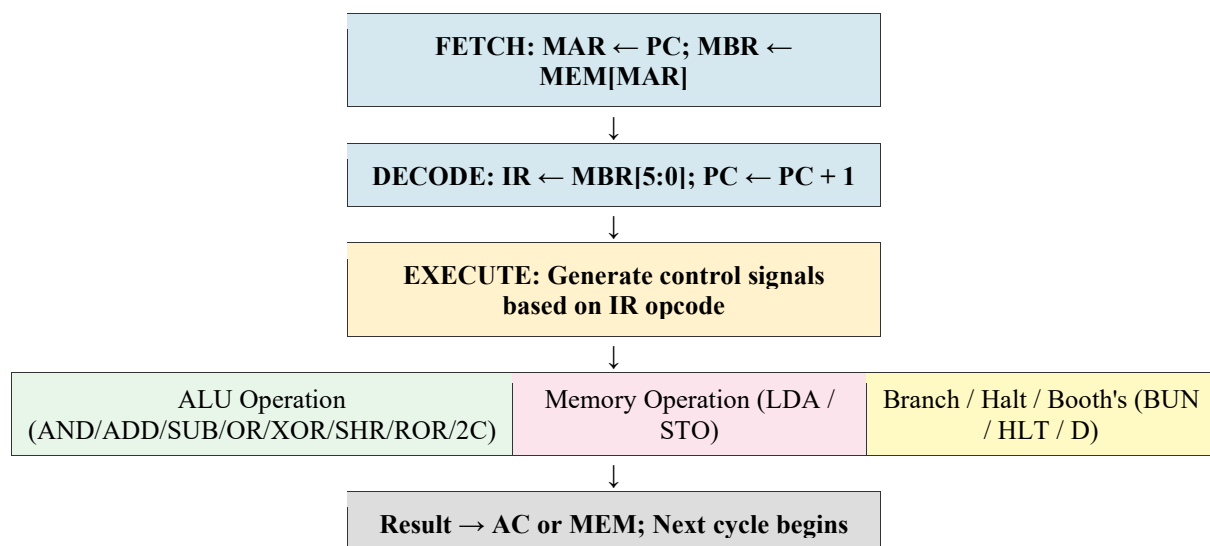
The CPU datapath includes the following key registers and buses:

- AC (Accumulator) — 23-bit register holding current computation result
- MBR (Memory Buffer Register) — holds data read from/written to RAM
- MAR (Memory Address Register) — holds the address for memory access
- IR (Instruction Register) — holds the current instruction
- PC (Program Counter) — embedded in the Control Unit's sequencer
- BOOTHS_HIGH / BOOTHS_LOW — 23-bit registers for multiplication result

4. Control Unit

The Control Unit (CU) is a hardwired sequencer implemented in Logisim. It takes the 4-bit opcode from the IR and generates all necessary control signals for each clock cycle.

4.1 Fetch-Decode-Execute Cycle



Control Functions and Microoperations

This table outlines the relationship between the control signals and the hardware microoperations as specified in the processor circuit documentation.

Control Function	Microoperation	Functional Description
—	None	No operation / Idle state.
d_1	$MAR \leftarrow PC$	Transfers the address in the Program Counter to the Memory Address Register.
c_1	$PC \leftarrow PC + 1$	Increments the Program Counter to point to the next instruction address.
R	$B \leftarrow M$	Read Operation: Fetches data from memory (at the address in MAR) into Register B.
c_2	$PC \leftarrow B(AD)$	Updates the Program Counter with the address portion of Register B (used for jumps/branches).
d_2	$MAR \leftarrow B(AD)$	Transfers the address portion of Register B to the Memory Address Register.
a_0, a_1, a_2	$ALU_Sel \leftarrow \{a_2, a_1, a_0\}$	ALU Control: Selection bits that determine the current operation of the ALU (e.g., ADD, SUB, AND, OR).
b_1	$B \leftarrow A$	Transfers the contents of Register A into Register B.
W	$M \leftarrow B$	Write Operation: Stores the contents of Register B into memory at the MAR address.
LdAfrB	$A \leftarrow B$	Loads Register A directly from the contents of Register B.
bth	$Result \leftarrow A \times B$	Booth's Multiplier: Initiates the Booth multiplication algorithm using A and B.
low	$AC_{low} \leftarrow Result_{0..22}$	Stores the lower 23 bits of the Booth multiplication result.
high	$AC_{high} \leftarrow Result_{23..45}$	Stores the higher bits of the Booth multiplication result.
HLT	None	Halt: Terminates the execution of the current program.

5. Arithmetic Logic Unit (ALU)

The 23-bit ALU is implemented as a hierarchical sub-circuit within Logisim. It receives two 23-bit operands (A and B), a 3-bit control signal, and outputs a 23-bit result along with status flags.

5.1 ALU Operations and Control Encoding

Control[2:0]	Operation	Notes
000	AND	Bitwise AND — uses 23-bit AND sub-circuit
001	ADD	23-bit ripple-carry adder
010	OR	Bitwise OR — uses 23-bit OR sub-circuit
011	SUB	Subtraction via ADD/SUB module (2's complement B)
100	XOR	Bitwise XOR
101	SHR	Logical right shift by 1
110	ROR	Rotate right; MSB receives LSB
111	2's Comp	Negation: $\sim A + 1$ using 2s_COMP sub-circuit

5.2 Status Flags

Flag	Meaning
Zero	Result is all zeros (computed via Reduce-OR gate on all 23 bits)
Negative	MSB (bit 22) of result is 1
Overflow	Signed overflow detected in addition/subtraction
Carry	Carry-out from the MSB of the adder

5.3 ALU Sub-circuits

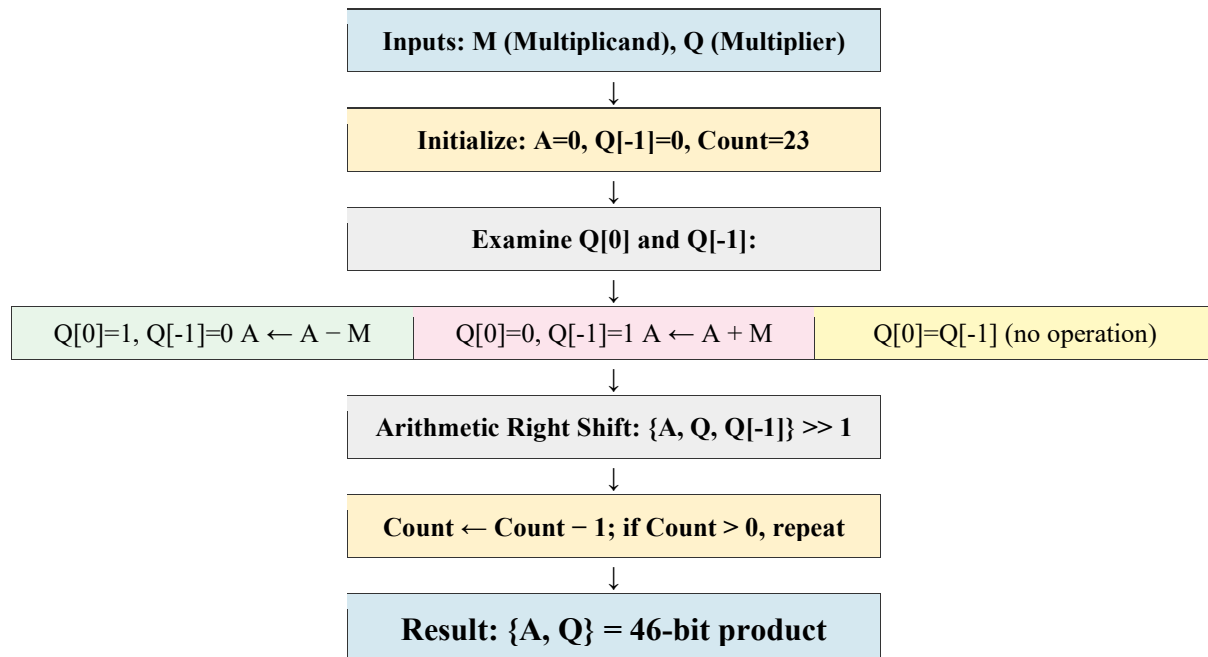
The ALU is built from the following reusable sub-circuits:

1. **Full_adder** — 1-bit full adder (building block)
2. **23_bit_full_adder** — chain of 23 full adders with carry propagation
3. **23_bit_full_adder_concise** — optimized version of above
4. **23_bit_and** / **23_bit_or** / **23_bit_xor** — bitwise logic units
5. **Add_sub** — 23-bit add/subtract (XOR B with subtract flag + carry-in)
6. **SHR** / **ROR** — shift and rotate right units
7. **2s_COMP** — 23-bit two's complement unit
8. **23_bit_reduce_or** — 23-input OR tree for Zero flag
9. **22th bit out** — extracts MSB for Negative flag

6. Booth's Multiplier

The Booth's Multiplier implements the standard Booth's algorithm to multiply two 23-bit signed integers. The result is a 46-bit product split into two 23-bit halves (BOOTHHS_HIGH and BOOTHHS_LOW), accessible by the CPU via opcodes C and D.

6.1 Booth's Algorithm Overview



6.2 Hardware Components

Component	Role
M Register (23-bit)	Holds the multiplicand; loaded from CPU via BOOTHHS_MUL instruction
Q Register (23-bit)	Holds the multiplier; shifts right each iteration
A Register (23-bit)	Accumulator for partial products
23-bit ALU	Handles add/subtract within Booth's iterations
Controller	Finite state machine controlling step count (17 steps shown in simulation)
Shift Network	Arithmetic right shift across {A, Q, Q[-1]} triple
HALT output	Signals completion to the CPU (stall released)
Higher / Lower outputs	23-bit upper and lower halves of the product

6.3 CPU Integration

Booth's multiplier is integrated into the CPU as a co-processor. When the CPU decodes opcode D (BOOTHS_MULTIPLICATION):

10. The CPU asserts the bth signal, passing AC and B register values as M and Q inputs.
11. The CPU enters STALL state (highBTH/lowBTH signals freeze the datapath).
12. Booth's multiplier runs its iterative hardware loop and asserts halt when done.
13. BOOTHS_LOW is available via the lower-bths bus.

7. Memory Subsystem

The main memory is a 4×6 RAM component visible in the main circuit, displaying four 6-bit memory words at addresses 00–03. It is connected to the CPU via a bidirectional data bus and controlled by the str (store), sel (select), ld (load), and clr (clear) signals.

Signal	Direction	Description
W	CPU → RAM	Store: write signal for MBR contents to MAR address
Data_out	CPU → RAM	Data to write to RAM
R	RAM → CPU	Load: put RAM[MAR] onto data bus
Data_in	RAM → CPU	Data to read from RAM
A / D buses	Bidirectional	23-bit data bus (A=address side, D=data side)

8. Simulation and Testing

The project was simulated in Logisim using the built-in clock stepping and manual reset functionality. Memory was pre-loaded with instruction words to test the fetch-execute cycle.

8.2 Test Observations

- Memory initializes to 000000 on all addresses at reset.
- ALU Zero/Negative/Carry/Overflow flags respond correctly for ADD and SUB operations.
- Booth's controller iterates 23 steps (counter visible in simulation), then asserts halt.
- STALL line correctly freezes PC advancement during Booth's multiplication.
- HLT instruction (opcode 7) correctly activates STALL output indefinitely.

9. Module Hierarchy Summary

Module	Sub-components
main	CPU, RAM (4×6), Reset, CLK
CPU	ControlUnit, 23-bit ALU, MAR, MBR, AC, IR, MUXes, BOOTHs_HIGH/LOW registers
ControlUnit	Instruction decoder (Decd), ROM-based microcode, PC register, priority logic
23-bit ALU	23_bit_and, 23_bit_OR, Add_sub, XOR, SHR, ROR, 2s_COMP, Reduce-OR, 22th bit out
23bit_Booths_multiplication	Booth's multiplier, 4-bit ALU, Controller, M/Q/A registers, shift network

11. Future Improvements

While the current 23-bit CPU architecture successfully implements a functional Instruction Set Architecture (ISA) and integrated Booth's Multiplier, several enhancements could further optimize performance, scalability, and usability. The following areas are identified for future development:

11.1 Architectural Pipelining

The current design utilizes a multi-cycle execution model where the CPU must complete one instruction before fetching the next. Implementing a 3-stage or 5-stage pipeline (Fetch, Decode, Execute, Memory, Write-back) would significantly increase throughput. This would require the implementation of hazard detection units to handle data and control hazards, particularly for branch instructions (BUN).

11.2 Expansion of Memory Address Space

The current 5-bit address field limits the system to 32 memory locations. This is a significant bottleneck for complex programs. Future iterations could utilize the 11 "Don't Care" bits in the instruction format to expand the address bus. Transitioning to a 12-bit or 16-bit address space would allow the CPU to interface with much larger RAM modules, enabling more sophisticated software applications.

11.3 Enhanced Instruction Set (ISA 2.0)

The current 13-instruction set provides a solid foundation but lacks several features found in modern RISC/CISC architectures:

- **Conditional Branching:** Adding instructions like BEZ (Branch if Zero) or BNE (Branch if Not Equal) would allow for complex logic loops and conditional execution.
- **Immediate Addressing:** Currently, all operations require memory access. Adding support for immediate values (e.g., ADDI AC, #value) would improve execution speed for constants.
- **Stack Operations:** Implementing PUSH and POP instructions, along with a dedicated Stack Pointer (SP) register, would facilitate nested function calls and recursive algorithms.

The ALU currently relies on a ripple-carry adder, which introduces $O(n)$ gate delay. Upgrading this to a Carry-Lookahead Adder (CLA) would reduce delay to $O(\log n)$, allowing for higher clock frequencies. Additionally, the Booth's Multiplier could be enhanced with a Wallace Tree structure to perform partial product addition in parallel, reducing the total cycle count for multiplication.

11.5 Input/Output (I/O) Interfacing

To transition from a theoretical simulation to a more practical system, the CPU needs a standardized I/O mapping. Implementing Memory-Mapped I/O would allow the CPU to interact with external components such as 7-segment displays, keyboards, or UART modules within Logisim, turning the CPU into a complete microcontroller system.

11.6 Software Toolchain Development

Currently, instructions must be manually encoded into hex. Developing a basic Assembler (using Python or C++) that converts mnemonic code (e.g., LDA 0x05) into the CPU's 23-bit binary format would greatly simplify program development and testing.

11.7 Interrupt Handling

Implementing an interrupt controller would allow the CPU to handle asynchronous external events. This involves adding an Interrupt Enable (IE) flag, an Interrupt Vector Table (IVT), and logic to save the current Program Counter (PC) and Accumulator (AC) to a stack before branching to an Interrupt Service Routine (ISR).

12. Conclusion

The 23-bit CPU project serves as a comprehensive demonstration of digital logic design and computer architecture principles implemented within the Logisim environment. By integrating custom-built components into a cohesive system, the project achieves a fully operational processing unit capable of executing complex computational tasks.

The success of the design is highlighted by the following key accomplishments:

- **Robust Instruction Set Architecture (ISA):** The implementation of a 13-instruction ISA provides a versatile foundation, supporting essential operations including arithmetic, bitwise logic, memory access (load/store), unconditional branching, and specialized shift/rotate functions.
- **High-Precision Hierarchical ALU:** The system features a 23-bit ALU capable of 8 distinct operations, utilizing a modular design built from gate-level sub-circuits. The inclusion of four status flags—Zero, Negative, Carry, and Overflow—enables the CPU to monitor and respond to the mathematical state of computations in real-time.
- **Sophisticated Control Logic:** A hardwired Control Unit manages the system's timing and sequencing. By utilizing microcode-based ROM sequencing, the unit efficiently coordinates the multi-cycle fetch-decode-execute process, ensuring synchronized data flow between registers and memory.
- **Integrated Booth's Multiplication Co-processor:** A standout feature of the architecture is the hardware-accelerated Booth's Multiplier. This co-processor handles 23-bit signed multiplication through a specialized iterative loop, utilizing a STALL mechanism to freeze the main CPU datapath until the high-fidelity 46-bit product is calculated.
- **Verified Simulation and Reliability:** Extensive testing within Logisim has confirmed the accuracy of the system's execution cycles. Verification includes the correct handling of ALU flags, the successful execution of the 23-step Booth's iteration, and the proper functioning of the HLT instruction to terminate processing safely.